

Raport z listy 10

Modele regresji i ich zastosowania

Ksawery Józefowski, album 277513

2026-05-13

Spis treści

1	Generowanie zbioru danych do regresji logistycznej	1
1.1	Część 1.	1
1.2	Część 2.	2
2	Konstrukcja modelu regresji logistycznej	3
2.1	Zadanie 1: Wczytać do pakietu statystycznego dane z utworzonego pliku. . .	3
2.2	Zadanie 2	6
2.3	zadanie 3	6
2.4	Zadanie 4	8

1 Generowanie zbioru danych do regresji logistycznej

Model regresji logistycznej jest postaci:

$$\log \frac{p(\mathbf{x})}{1 - p(\mathbf{x})} = x\beta^T$$

gdzie $p(\mathbf{x}) = \mathbb{P}(Y = 1 \mid \mathbf{X} = \mathbf{x})$ to prawdopodobieństwo sukcesu, $\mathbf{x}$ to wektor cech, a β to wektor nieznanych parametrów w modelu.

Reguła klasyfikacyjna jest postaci:

$$d^*(\mathbf{x}) = \begin{cases} 1, & \text{gdy } p(\mathbf{x}) > \pi_0, \\ 0, & \text{gdy } p(\mathbf{x}) \leq \pi_0, \end{cases}$$

gdzie π_0 jest pewną stałą z przedziału $(0, 1)$.

1.1 Część 1.

Generujemy $n = 300$ obserwacji a liczba zmiennych objaśniających jest równa 20.

```
set.seed(123)
n <- 300
p <- 20
beta <- rep(0, p)
beta[1:10] <- c(1, -2, 1.5, -1, 2, -1.5, 0.8, -0.8, 1.2, -1.2)
```

Współczynniki β w modelu to

```
print(cbind(i=1:20, beta_i=beta))
```

```
##      i beta_i
## [1,] 1  1.0
## [2,] 2 -2.0
## [3,] 3  1.5
## [4,] 4 -1.0
## [5,] 5  2.0
## [6,] 6 -1.5
## [7,] 7  0.8
## [8,] 8 -0.8
## [9,] 9  1.2
## [10,] 10 -1.2
## [11,] 11  0.0
## [12,] 12  0.0
## [13,] 13  0.0
## [14,] 14  0.0
## [15,] 15  0.0
## [16,] 16  0.0
## [17,] 17  0.0
## [18,] 18  0.0
## [19,] 19  0.0
## [20,] 20  0.0
```

1.2 Część 2.

1.2.1 Podpunkt (a)

Dla $i = 1, 2, \dots, 20$ przyjmujemy, że $x_{i1} = 1$ a następnie generujemy, za pomocą standardowego rozkładu normalnego $\mathcal{N}(0, 1)$, wartości $x_{i2}, x_{i3}, \dots, x_{i20}$ dla pozostałych $p - 1 = 19$ zmiennych objaśniających $X_2, X_3, \dots, X_{20}$

```
set.seed(123)
X1 <- rep(1, n)
X2...20 <- matrix(rnorm(n * (p - 1)), nrow = n, ncol = p - 1)
X <- cbind(X1, X2...20)
```

1.2.2 Podpunkt (b)

Generujemy wartości $y_i, i = 1, 2, \dots, 300$ binarnej zmiennej objaśniającej Y , która przyjmuje wartości 0 oraz 1 z prawdopodobieństwami, odpowiednio, $\pi(\mathbf{x}_i)$ oraz $1 - \pi(\mathbf{x}_i)$, gdzie $\mathbf{x}_i = (x_{i1}, \dots, x_{ip})$ oraz

$$\pi(\mathbf{x}_i) = \mathbb{P}(Y = 1 \mid X_1 = x_{i1}, \dots, X_p = x_{ip}) = \frac{1}{1 + \exp(-(\beta_1 x_{i1} + \dots + \beta_p x_{ip}))}$$

```
set.seed(123)
lin_pred <- X %*% beta
prob <- 1 / (1 + exp(-lin_pred))
Y <- rbinom(n, size = 1, prob = prob)
df <- as.data.frame(cbind(X, Y))
colnames(df) <- c(paste0("X", 1:20), "Y")
write.csv(df, file = "dane_logistyczne.csv", row.names = FALSE)
dane <- read.csv(file = "dane_logistyczne.csv", sep = ",", dec = ".", check.names = FALSE)
```

2 Konstrukcja modelu regresji logistycznej

2.1 Zadanie 1: Wczytać do pakietu statystycznego dane z utworzonego pliku.

2.1.1 podpunkt (a):

Cały zbiór danych zawiera ... obserwacji, które podzielimy w proporcji 70 : 30 na część uczącą i część testową za pomocą funkcji *createDataPartition()* z pakietu *caret*.

```
set.seed(123)
library(caret)
trainIndex <- createDataPartition(dane$Y, p = 0.7, list = FALSE)
train <- dane[ trainIndex, ]
test <- dane[-trainIndex, ]
```

2.1.2 podpunkt (b):

Modele regresji logistycznej stworzymy przy pomocy funkcji *glm()*.

```
model.full <- glm(Y ~ ., data = train[, -1], family = binomial)
model.null <- glm(Y ~ 1, data = train, family = binomial)
```

2.1.3 podpunkt (c):

Do przeprowadzenia wspomnianego w zadaniu testu ilorazu wiarygodności wykorzystamy funkcję *anova()*.

```
lrt.test <- anova(model.null, model.full, test = "LRT")
p.value <- lrt.test$`Pr(>Chi)`[2]
```

Na podstawie testu opartego na ilorazie wiarygodności na poziomie istotności $\alpha = 0.05$ **odrzucaamy** hipotezę H_0 mówiącą, że $\beta_2 = \beta_3 = \dots = \beta_{20} = 0$, więc przynajmniej jedna ze zmiennych objaśniających $X_2, \dots, X_{20}$ ma istotny wpływ na Y . P-wartość tego testu wynosi $4.31e - 38$.

2.1.4 podpunkt (d):

Estymatory $\hat{\beta}_i$ oraz ich błędy standardowe SE_{β_i} otrzymujemy za pomocą podsumowania `summary()` modelu pełnego i są podane w tabeli poniżej.

```
summary(model.full)[["coefficients"]]
```

##	Estimate	Std. Error	z value	Pr(> z)
## (Intercept)	2.3388063	0.6990681	3.3456059	0.0008210300
## X2	-4.8239831	1.3147739	-3.6690591	0.0002434448
## X3	4.1260695	1.1459051	3.6007080	0.0003173519
## X4	-2.4352315	0.6931594	-3.5132342	0.0004426870
## X5	3.9627110	1.0880413	3.6420592	0.0002704658
## X6	-3.8014000	1.0895369	-3.4890052	0.0004848218
## X7	1.3927426	0.5042753	2.7618697	0.0057471411
## X8	-1.7901641	0.6256880	-2.8611133	0.0042215611
## X9	2.8459762	0.8754739	3.2507836	0.0011508742
## X10	-3.9675733	1.0912680	-3.6357461	0.0002771772
## X11	-0.6733681	0.3855468	-1.7465273	0.0807193632
## X12	0.3884958	0.3776387	1.0287502	0.3035970954
## X13	-0.0660561	0.3600958	-0.1834404	0.8544525027
## X14	-0.7081982	0.4773550	-1.4835881	0.1379182227
## X15	-0.3594191	0.3742756	-0.9603060	0.3369012141
## X16	-0.3087401	0.3619221	-0.8530567	0.3936278571
## X17	1.0367991	0.5042893	2.0559611	0.0397862630
## X18	-1.2520429	0.5615660	-2.2295561	0.0257769299
## X19	0.2505089	0.4428534	0.5656701	0.5716180672
## X20	-0.8815244	0.5235250	-1.6838250	0.0922154977

Przedziały ufności otrzymujemy za pomocą funkcji `confint()` i są podane w kolejnej tabeli.

```
set.seed(123)
przedzialy <- confint(model.full, level = 0.95)
przedzialy
```

##	2.5 %	97.5 %
## (Intercept)	1.1850427	3.98095507
## X2	-8.0748182	-2.76993977
## X3	2.3398172	6.97416060

```
## X4          -4.0926191 -1.30211604
## X5           2.2711209  6.68832705
## X6          -6.4733374 -2.08324490
## X7           0.5175187  2.55423738
## X8          -3.2287403 -0.72886208
## X9           1.4532900  4.95405617
## X10         -6.7545997 -2.28094350
## X11         -1.5099860  0.04488541
## X12         -0.3124930  1.21157637
## X13         -0.7945753  0.65214924
## X14         -1.7652147  0.15490199
## X15         -1.1373484  0.36519011
## X16         -1.0617285  0.39299049
## X17          0.1302434  2.15211631
## X18         -2.5094238 -0.25728953
## X19         -0.6283342  1.13674169
## X20         -2.0586564  0.05108932
```

Do zweryfikowania hipotezy $H_0 : \beta_i = 0$ przeciwko hipotezie alternatywnej $H_1 : \beta_i \neq 0$ można wykorzystać tabelę z przedziałami ufności.

Otrzymujemy z niej, że zmienne $X_2, X_3, X_4, X_5, X_6, X_7, X_8, X_9, X_{10}, X_{17}, X_{18}$ mają istotny wpływ na zmienną Y , ponieważ ich przedziały ufności nie zawierają zera. Zmienne $X_{11}, X_{12}, X_{13}, X_{14}, X_{15}, X_{16}, X_{19}, X_{20}$ są nieistotne, gdyż odpowiadające im przedziały ufności zawierają wartość zero.

Alternatywnie: można wykorzystać wyznaczone p-wartości tych testów:

```
which(summary(model.full)[["coefficients"]][, 4] < 0.05)
```

```
## (Intercept)      X2      X3      X4      X5      X6
##           1         2         3         4         5         6
##           X7      X8      X9     X10     X17     X18
##           7         8         9        10        17        18
```

2.1.5 podpunkt (e):

```
data.frame(beta = beta,
            beta.est = model.full$coefficients,
            istotny = ifelse(summary(model.full)[["coefficients"]][, 4] < 0.05,
                             "TAK",
                             "nieistotna"))
```

```
##      beta  beta.est  istotny
## (Intercept)  1.0  2.3388063    TAK
## X2          -2.0 -4.8239831    TAK
## X3           1.5  4.1260695    TAK
```

## X4	-1.0	-2.4352315	TAK
## X5	2.0	3.9627110	TAK
## X6	-1.5	-3.8014000	TAK
## X7	0.8	1.3927426	TAK
## X8	-0.8	-1.7901641	TAK
## X9	1.2	2.8459762	TAK
## X10	-1.2	-3.9675733	TAK
## X11	0.0	-0.6733681	nieistotna
## X12	0.0	0.3884958	nieistotna
## X13	0.0	-0.0660561	nieistotna
## X14	0.0	-0.7081982	nieistotna
## X15	0.0	-0.3594191	nieistotna
## X16	0.0	-0.3087401	nieistotna
## X17	0.0	1.0367991	TAK
## X18	0.0	-1.2520429	TAK
## X19	0.0	0.2505089	nieistotna
## X20	0.0	-0.8815244	nieistotna

Estymator $\hat{\beta}$ poprawnie zidentyfikował wszystkie niezerowe elementy wektora β (zmienne $X_2, \dots, X_{10}$) jako istotne i prawidłowo określił ich znaki. Wśród zmiennych o zerowych współczynnikach błędnie uznał za istotne X_{17} oraz X_{18} , co przy poziomie istotności $\alpha = 0.05$ i 10 testach jest zjawiskiem naturalnym.

2.2 Zadanie 2

Przykładowo, dla zmiennych X_3 oraz X_6 te ilorazy szans to, odpowiednio, $\exp(\hat{\beta}_3) \approx 61.934015$ oraz $\exp(\hat{\beta}_6) \approx 0.0223395$.

Oznacza to, że przy wzroście X_3 o 1 szanse zajścia zdarzenia $Y = 1$ **rosną** około 61.934015 krotnie, natomiast wzrost X_6 o jeden powoduje **spadek** szans o około 97.8% (tzn. szanse stają się 0.0223395 razy mniejsze).

Jeśli obie te zmienne wzrosną o 1 to wtedy szanse zmieniają się $61.934 \times 0.022 \approx 1.362$ krotnie, bo ich wspólny iloraz szans to iloczyn indywidualnych ilorazów szans, co oznacza nieznaczny **wzrost** szans o około 36.2%.

2.3 zadanie 3

Konstruujemy macierz pomyłek postaci

		Actual	
		0	1
Predicted	0	TN	FN
	1	FP	TP

Wyznaczamy miary zdolności predykcyjnej modelu:

- $Sensitivity = \frac{TP}{TP+FN}$
- $Specificity = \frac{TN}{TN+FP}$
- $FPR = \frac{FP}{TN+FP}$
- $FNR = \frac{FN}{TP+FN}$
- $Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$

```
predict.test <- predict(model.full, newdata = test, type = "response")

conf.matrix.metrics <- function(threshold) {
  pred.class <- ifelse(predict.test > threshold, 1, 0)
  cm <- table(Predicted = pred.class, Actual = test$Y)

  TP <- cm["1", "1"]; TN <- cm["0", "0"]
  FP <- cm["1", "0"]; FN <- cm["0", "1"]

  sensitivity <- TP / (TP + FN)
  specificity <- TN / (TN + FP)
  fpr <- FP / (TN + FP)
  fnr <- FN / (TP + FN)
  acc <- (TP + TN) / sum(cm)

  list(cm = cm,
        sensitivity = sensitivity,
        specificity = specificity,
        fpr = fpr,
        fnr = fnr,
        accuracy = acc)
}

M.1 <- conf.matrix.metrics(0.5)
M.2 <- conf.matrix.metrics(0.7)
```

Macierze pomyłek:

```
print(M.1$cm)
```

```
##           Actual
## Predicted  0   1
##           0 30  4
##           1  5 51
```

```
print(M.2$cm)
```

```
##           Actual
## Predicted  0   1
##           0 31   7
##           1   4 48
```

Wskaźniki porównujące modele:

```
data.frame(
  Miara = c("Sensitivity", "Specificity", "FPR", "FNR", "Accuracy"),
  pi0_05 = round(c(M.1$sensitivity, M.1$specificity,
                  M.1$fpr, M.1$fnr, M.1$accuracy), 4),
  pi0_07 = round(c(M.2$sensitivity, M.2$specificity,
                  M.2$fpr, M.2$fnr, M.2$accuracy), 4)
)
```

```
##           Miara pi0_05 pi0_07
## 1 Sensitivity 0.9273 0.8727
## 2 Specificity 0.8571 0.8857
## 3           FPR 0.1429 0.1143
## 4           FNR 0.0727 0.1273
## 5     Accuracy 0.9000 0.8778
```

Dla progu $\pi_0 = 0.5$ model osiąga wyższą czułość oraz ogólną dokładność, kosztem niższej swoistości. Dla progu $\pi_0 = 0.7$ swoistość wzrasta, a FPR spada, jednak model częściej pomija prawdziwe przypadki $Y = 1$. W obu przypadkach model radzi sobie bardzo dobrze — wybór progu zależy od tego, czy ważniejsze jest unikanie fałszywie dodatnich czy fałszywie ujemnych klasyfikacji.

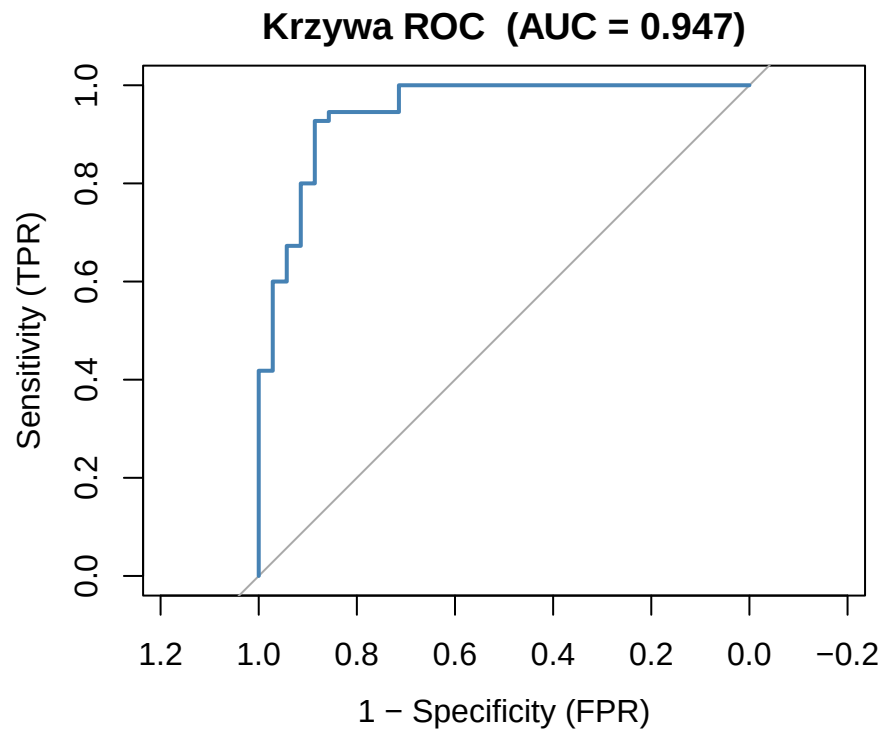
Dla $\pi_0 = 0.5$ model popełnił 5 błędy FP i 4 FN, natomiast dla $\pi_0 = 0.7$ odpowiednio 4 FP i 7 FN, wyższy próg zmniejsza FN kosztem nieznacznego wzrostu FP.

2.4 Zadanie 4

```
library(pROC)
roc.object <- roc(test$Y, predict.test)
auc(roc.object)
```

```
## Area under the curve: 0.947
```

```
plot(roc.object,
     col = "steelblue",
     lwd = 2,
     main = paste0("Krzywa ROC (AUC = ", round(auc(roc.object), 3), ")"),
     xlab = "1 - Specificity (FPR)",
     ylab = "Sensitivity (TPR)")
```

Pole pod krzywą ROC wynosi $AUC = 0.947$, co oznacza, że model z prawdopodobieństwem 94.7% przypisze wyższe prawdopodobieństwo sukcesu losowo wybranej obserwacji z klasy $Y = 1$ niż losowo wybranej obserwacji z klasy $Y = 0$. Wartość ta jest bliska 1, co świadczy o bardzo dobrej zdolności predykcyjnej modelu.